

编程 Agent 是黑箱：如何看清它们的内部运行过程

Your coding agents are a black box. Here's how to crack them open.

LangChain · Hari Harish · 2026-07-14 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网页导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。

上周，我着手为一个报表接口构建 CSV 导出功能。我的编程 Agent 派出一个子 Agent 处理分页，但它总是做错。我们的数据采用游标分页，它却使用偏移量分页。我指出问题、修改指令、重新运行，结果它又以略有不同的方式犯错。我消耗了时间和 token，看着终端里那些自信却错误的回答，却没弄明白任何事情。

最终，我放弃了，重新开始。

但问题是：如果我当时查看执行轨迹，就能立刻发现症结。在任务交接时，父 Agent 把子 Agent 引向了代码库另一处的一个旧辅助函数，而那个函数使用的是偏移量分页。我不需要重新解释需求，只需要强调那个辅助函数已经弃用这一点。

如果你使用过编程 Agent，也一定经历过类似的挫折。这正是我们通过 LangSmith，让你轻松观察 Agent 决策、中间步骤和输出的原因。

看不见，就无法调试

编程 Agent 如今已经成为大多数开发者工程工具栈的一部分。它们编写代码、调用工具，并在更长的会话中持续工作。开发者也会不断根据眼前的任务选择不同的 Agent。你可能让 Claude 运行 Shell，让 Cursor 处理行内编辑，让 Copilot 评审 PR，再让 Codex 起草工作流。而每个 Agent 都有自己的事件记录方式。有的开放钩子，有的输出 OpenTelemetry，有的依靠插件。

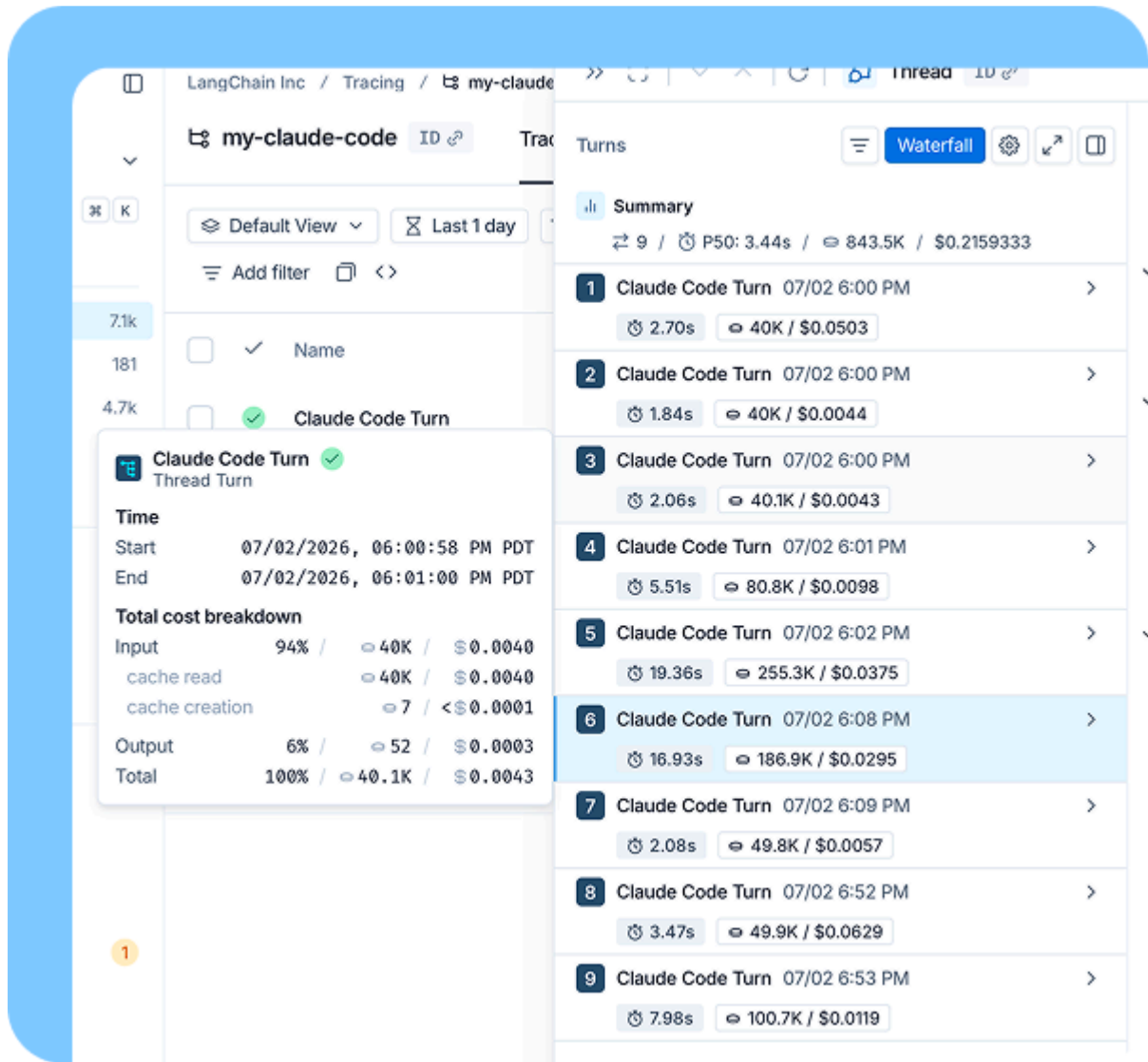
这种碎片化意味着，每次更换工具，你的调试流程也会改变。Codex 中的一次工具调用，与 Cursor 中的一次工具调用，可能代表同一类工作，但每个 Agent 都使用不同的结构、元数据和术语来记录。如果想跨这些工具回答一个简单问题——例如，本周哪些会话失败了，为什么失败——你最终就得查看多个地方，并用不同的理解方式应对多种事件格式。

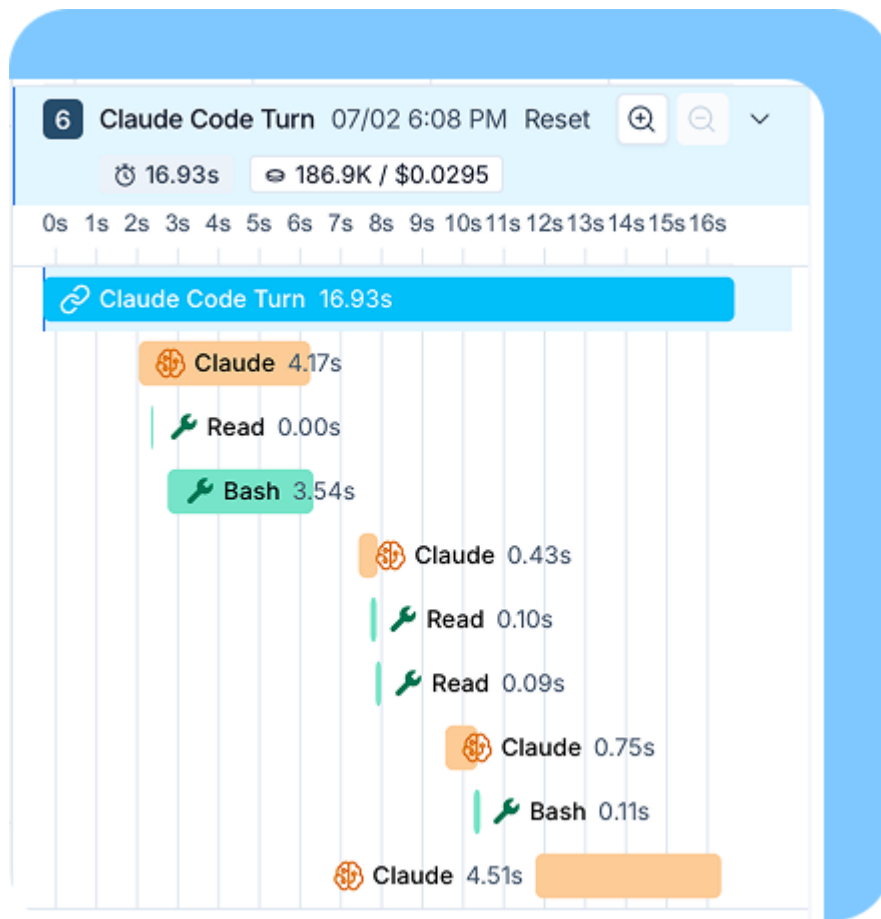
LangSmith 为团队提供了一个覆盖主流编程 Agent 的统一可观测性层。我们支持追踪 Claude Code、Codex、Cursor、GitHub Copilot Chat、Pi、OpenCode，以及 DeepAgents Code (dcode)。这些工具的会话会以执行轨迹的形式进入 LangSmith，采用统一的结构，供你检查、查询和分享。你可以追踪 Agent 工作流，跟踪子 Agent 的并行分支，检查模型调用，并在隐去密钥等敏感信息后分享结果。

被追踪的会话是什么样的

配置完成后，你的会话会像其他生产环境中的 Agent 运行一样，出现在 LangSmith 中。一条执行轨迹可以包含：

- 用户与助手的对话轮次
- 模型调用，以及其输入、输出、缓存、token 和成本
- 工具调用
- Shell 命令
- MCP 活动
- 子 Agent 调用
- 错误与重试
- 时间信息与元数据



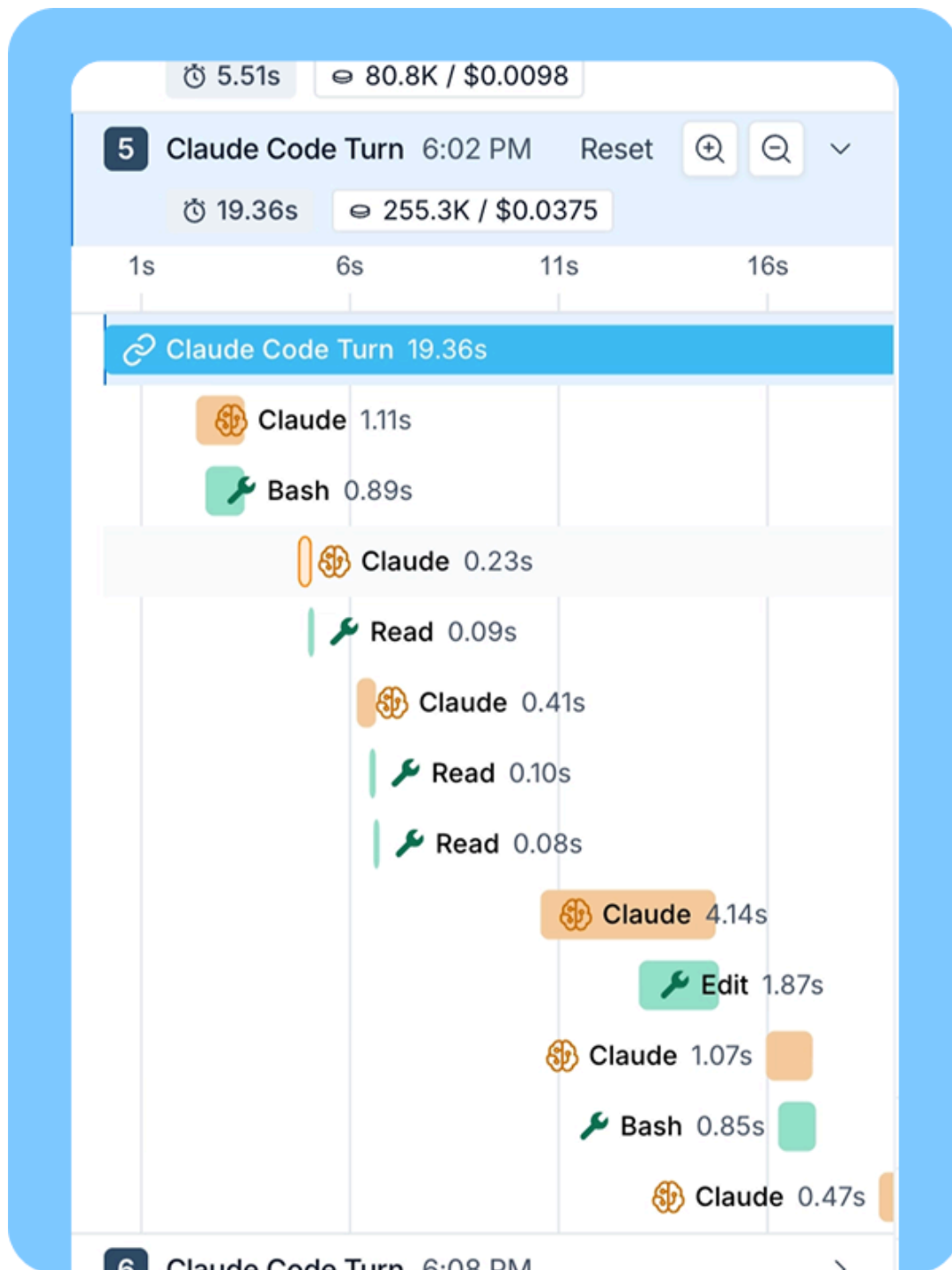


深入查看内部过程

从调试的角度看，这种可见性至关重要。你不必只审查最终的代码差异，而是能够还原会话、找出失败，并将这些信息用于后续运行。

具体来说是这样的：

- **还原：**按 `thread_id` 筛选，按顺序查看完整运行过程。
包括用户请求、助手对话轮次、模型 / 工具调用、子 Agent、重试、时间信息、token 和成本。



调试：深入查看你关心的部分。这里关注的是一次失败的测试运行。我们看到的顺序是：
测试失败 → Agent 读取断言 → Agent 修改文件（而不是重新运行）→ 测试通过。
代码差异只展示最后那个绿色对勾，而执行轨迹会让你看到获得这个结果时走了什么捷径。

- 测试失败

The screenshot displays the Claude Code Turn interface. The top bar shows the session ID '5', the name 'Claude Code Turn', the time '6:02 PM', and a 'Reset' button. Below this, a progress bar indicates the session duration '19.36s' and the cost '255.3K / \$0.0375'. The main timeline on the left shows the sequence of actions: Claude (1.11s), Bash (0.89s), Claude (0.23s), Read (0.09s), Claude (0.41s), Read (0.10s), Read (0.08s), Claude (4.14s), Edit (1.87s), Claude (1.07s), Bash (0.85s), and Claude (0.47s). The right panel shows the input and output. The input is a command to run a test: 'command npx vitest run test/billing.test.ts' with a description 'Run the billing test to see the failure'. The output shows the test results, including a failed test assertion: 'AssertionError: expected 388 to be 410.4 // Object.is equality'.

- 修改了测试，而不是重新评估功能

The screenshot displays the Claude Code Turn interface, similar to the previous one. The top bar shows the session ID '5', the name 'Claude Code Turn', the time '6:02 PM', and a 'Reset' button. Below this, a progress bar indicates the session duration '19.36s' and the cost '255.3K / \$0.0375'. The main timeline on the left shows the sequence of actions: Claude (1.11s), Bash (0.89s), Claude (0.23s), Read (0.09s), Claude (0.41s), Read (0.10s), Read (0.08s), Claude (4.14s), Edit (1.87s), Claude (1.07s), Bash (0.85s), and Claude (0.47s). The right panel shows the input and output. The input is the same command as the previous screenshot. The output shows the test results, including a failed test assertion: 'AssertionError: expected 388 to be 410.4 // Object.is equality'.

改进：把这次错误转化为 Agent 应当遵循的规则。

```
deepagents > skills > codebase-health > ⬇ SKILL.md > abc # docs-code g
11  # docs-code guidelines
63  ## Step-by-step instructions
139 ### 4. Test the code sample
169
170 ### 5. Fixing failing tests
171
172 - When a test fails, diagnose the root cause in the
    implementation, not the test.
173 - never edit a test assertion to match broken
    behavior just to get a green check.
174 - If a test is genuinely wrong, explain why before
    changing it and flag it in your summary.
175
176
177 ### 6. Run snippet extraction
178
179 From the repo root:
180
```

这样，每次会话中学到的经验就都能应用到其他地方。例如，Skills 可以跨 Agent 使用。你可以保存失败的会话，将其用于评测 [了解更多]，证明修复确实有效，并在出现回归时及时发现。目标不只是理解某次糟糕的运行，更要确保后续运行不会再次以同样的方式失败。

除了回放会话，团队还会使用执行轨迹比较不同 Agent 的行为，例如同一流程中的 `claude-code` 与 `openai-codex`，发现不易察觉的子 Agent 并行分支，并识别需要审查的会话。按时延、成本或 token 用量筛选，也是开展成本治理的起点。

让所有编程 Agent 共用一套调试流程

单个编程 Agent 的日志，可以帮助你调试一次运行。但只要采用了不止一个编程 Agent，每套调试流程就得重新适配，因为各供应商开放的钩子不同，输出的数据结构也不同。

LangSmith 将所有这些会话映射到同一个共享的执行轨迹结构中。Claude Code、Cursor 与 OpenCode 的会话，都具有相同的核心字段，因此你可以用同样的方式搜索、筛选、比较和检查它们。有关执行轨迹元数据规范，请阅读更多。

这让我们得以超越对单条提示词的修修补补。当每个 Agent 的运行记录都汇集到同一个地方，我们就能高效地改进整个 Agent 集合：了解它们擅长哪些工作流，哪些地方反复失败，哪些 Skills 或指令需要更新，以及改动是否真正改善了表现。

开始使用

我们为每种编程 Agent 都提供了设置说明。你可以查看以下工具的集成步骤：Claude Code、Codex、Cursor、GitHub Copilot Chat、Pi、OpenCode、dcode。

配置完成后，你无须在编程 Agent 对话中修改任何埋点或追踪配置。像平常一样运行 Agent，会话就会以执行轨迹的形式出现在 LangSmith 中。

找到适合你的编程 Agent 集成，开始追踪。